

Welford's algorithm for weighted statistics

Andrey A. Efanov¹, Sergey A. Ivliev², Alexey G. Shagraev³

¹²³*Applied Math and Artificial Intelligence (AMAI) department
National Research University "MPEI"*

Moscow, Russian Federation

¹1134togo@gmail.com, ²siriusfrk@gmail.com, ³alex.shagraev@gmail.com

Abstract—The Welford method is a simple and effective way to calculate averages, variances, covariances, and other statistics. This method has several excellent properties: achieves high performance on the accuracy of decisions; it is straightforward to understand and implement; it is a one-pass online algorithm, which is extremely useful in some situations. Welford's original article published in 1962. Nevertheless, it cannot the algorithm is not widely known at present. Moreover, it is a non-trivial task to find a mathematical proof of its correctness or experimental comparisons with other methods. This article attempts to fill these gaps.

Index Terms—welford's online algorithm, weighted mean, variance, covariance, accuracy

I. INTRODUCTION

The calculation of the mean is one of the fundamental arithmetic problems used in various fields of mathematics. A rare program manages to avoid this task. Basic statistical algorithms depend on it (e.g., linear regression, kMeans, PCA [1]) and big data analytics programs [2].

It is also important task for modelling problems [3].

The more a program calculates the average, the more it depends on its accuracy. Many algorithms solve calculating the average and optimize speed, accuracy, memory consumption, and other characteristics. One of them is the Welford's algorithm, introduced in 1962 [4] – a one-pass online algorithm with high accuracy. It is easy to understand and implement and modify it to calculate variances, covariances and other statistics. Due to its properties, it can be parallelized and used in distributed computing.

This paper aims to provide a modification of Welford's algorithm for variance and covariance calculation, prove its correctness, and compare it to well-known alternatives, such as the Kahan summation algorithm. [5]

This paper has the following structure. Section 2 discovers the problem of mean calculation errors in computer systems. Section 3 describes the common designations of Welford's algorithm. Finally, the experimental results are shown in section 4.

II. NAIVE MEAN CALCULATION

The arithmetic mean is the sum of the sampled values divided by the number of items in the sample [6], [7]. Let's look at the following class that performs the naive floating-point mean calculation, written in C++.

```
class TDummyMeanCalculator {
```

```
private :  
    float SumValues = 0.;  
    size_t CountValues = 0;  
public :  
    void Add(const float value) {  
        ++CountValues;  
        SumValues += value;  
    }  
  
    double Mean() const {  
        return CountValues ?  
            SumValues / CountValues : 0.;  
    }  
};
```

It is easy to show that a simple program that calculates the mean of n numbers valued $1 * 10 - 3$ produces ratio errors. Table I shows the output of the program. The sum column added to increase the problem visibility. At some point, the error grows dramatically. This happens because of the digit capacity limit for the *float* data type.

TABLE I
NAIVE MEAN CALCULATION RESULT

#	Sample size	Sum	Mean
1	10000	10.0004	0.001000040
2	1000000	991.142	0.000991142
3	100000000	32768	0.000327680
4	200000000	32768	0.000163840
5	300000000	32768	0.000109227

There are several ways to solve this issue:

- use another data type (e.g.double instead of float),
- change the summation method (e.g., sorted, pairwise or Kahan summation), [5], [8], [9]
- use Welford's algorithm for mean calculation. [4]

It is not easy to find data for which a solution that uses double-precision floating-point numbers does not work well. However, such data is found, especially in more complex tasks. It is also unpleasant that using double increases memory costs: sometimes, one needs to store a large number of averages simultaneously.

Kahan's methods and other sophisticated summation methods are suitable in their way, but in the future, we will see that the Welford's algorithm still works better, and, besides,

it is surprisingly simple to implement. Indeed, let us look at the corresponding code:

```
class TWelfordMeanCalculator {
private:
    float MeanValues = 0.;
    size_t CountValues = 0;
public:
    void Add(const float value) {
        ++CountValues;
        MeanValues +=
            (value - MeanValues) /
                CountValues;
    }

    double Mean() const {
        return MeanValues;
    }
};
```

A similar experiment, shows its perfect accuracy.

TABLE II
WELFORD'S ALGORITHM MEAN CALCULATION RESULT

#	Sample size	Mean
1	10000	0.001
2	1000000	0.001
3	100000000	0.001
4	200000000	0.001
5	300000000	0.001

In II where is no sum column, because Welford's algorithm doesn't operate sum of values, but it gives us expected value of mean.

Also, these methods can be used in multivariate linear models,

III. WELFORD'S ALGORITHM FOR VARIANCE AND COVARIANCE CALCULATION

A. Designations

Assume that we have two sequences of floating point numbers: x and y , and a sequence of corresponding weights ω .

$$\begin{aligned} x &= x_1, x_2, \dots, x_n, \dots \\ y &= y_1, y_2, \dots, y_n, \dots \\ \omega &= \omega_1, \omega_2, \dots, \omega_n, \dots \end{aligned} \quad (1)$$

As soon as we need to calculate both means and covariances, we need to denote weighted sums and sums of products. We define it using a unified form, shown in equation 2.

$$S_n^{ab\dots z} = \sum_{i=1}^n a_i \cdot b_i \cdot \dots \cdot z_i \quad (2)$$

Thus S_n^ω denotes a sum of weights, $S_n^{\omega, x}$ – a weighted sum, and $S_n^{\omega, x, y}$ – a weighted sum of products.

$$\begin{aligned} S_n^\omega &= \sum_{i=1}^n \omega_i \\ S_n^{\omega x} &= \sum_{i=1}^n \omega_i \cdot x_i \\ S_n^{\omega xy} &= \sum_{i=1}^n \omega_i \cdot x_i \cdot y_i. \end{aligned} \quad (3)$$

We define weighted average values as shown in the following formula:

$$\begin{aligned} m_n^{\omega x} &= \frac{\sum_{i=1}^n \omega_i x_i}{\sum_{i=1}^n \omega_i} = \frac{S_n^{\omega x}}{S_n^\omega} \\ m_n^{\omega y} &= \frac{\sum_{i=1}^n \omega_i y_i}{\sum_{i=1}^n \omega_i} = \frac{S_n^{\omega y}}{S_n^\omega} \end{aligned} \quad (4)$$

Finally, we define unnormalized dispersion $D_n^{\omega xy}$ and unnormalized covariance $C_n^{\omega xy}$ as

$$D_n^{\omega xy} = \sum_{i=1}^n \omega_i (x_i - m_n^{\omega x})(y_i - m_n^{\omega y}) \quad (5)$$

$$C_n^{\omega xy} = \frac{D_n^{\omega xy}}{S_n^\omega} = \frac{\sum_{i=1}^n \omega_i (x_i - m_n^{\omega x})(y_i - m_n^{\omega y})}{\sum_{i=1}^n \omega_i} \quad (6)$$

B. Mean

Let's prove the formula for the weighted mean.

$$\begin{aligned} m_{n+1}^{\omega x} &= \frac{S_{n+1}^{\omega x}}{S_{n+1}^\omega} = \frac{S_n^{\omega x} + S_n^{\omega x} \omega_{n+1}}{S_n^\omega + S_n^{\omega x} \omega_{n+1}} \\ &= \frac{S_n^{\omega x} S_{n+1}^\omega + S_n^{\omega x} \omega_{n+1} S_n^\omega - S_n^{\omega x} S_n^\omega - \omega_{n+1} S_n^{\omega x} S_n^\omega}{S_n^\omega S_{n+1}^\omega} \\ &= \frac{S_n^{\omega x} S_{n+1}^\omega - \omega_{n+1} S_n^{\omega x} S_n^\omega}{S_n^\omega S_{n+1}^\omega} \\ &= \frac{\omega_{n+1} S_{n+1}^\omega - \omega_{n+1} S_n^\omega}{S_n^\omega S_{n+1}^\omega} \\ &= \frac{\omega_{n+1}}{S_{n+1}^\omega} \left(S_{n+1}^\omega - S_n^\omega \right) \\ &= \frac{\omega_{n+1}}{S_{n+1}^\omega} (x_{n+1} - m_n^{\omega x}). \end{aligned} \quad (7)$$

In particular when $\forall i \in 1..|\omega|. \omega_i = 1$ the following formula can be derived:

$$m_{n+1}^{\omega x} = m_n^{\omega x} + \frac{x_{n+1} - m_n^{\omega x}}{n+1} \quad (8)$$

C. Variance and covariance

We show that the classical formula for calculating covariance is also true for the weighted problem. Now for simplicity, we will work with non-normalized values.

$$\begin{aligned} D_n^{\omega xy} &= \sum_{i=1}^n \omega_i (x_i - m_n^{\omega x})(y_i - m_n^{\omega y}) \\ &= S_n^{\omega xy} - m_n^{\omega x} S_n^{\omega y} - S_n^{\omega x} m_n^{\omega y} + S_n^\omega m_n^{\omega x} m_n^{\omega y} \end{aligned} \quad (9)$$

Thus

$$D_n^{\omega xy} = S_n^{\omega xy} - \frac{S_n^{\omega x} S_n^{\omega y}}{S_n^{\omega}} \quad (10)$$

This formula is extremely convenient, including for the online algorithm, however, if the values are close and at the same time large in absolute value, its use will lead to significant computational errors.

Moreover, it is possible to derive a recurrence formula for a similar to the Wellford's formula for means.

$$\begin{aligned} D_{n+1}^{\omega xy} &= S_{n+1}^{\omega xy} + \omega_{n+1} x_{n+1} y_{n+1} \\ &\quad - \omega_{n+1} x_{n+1} \frac{S_{n+1}^{\omega y}}{S_{n+1}^{\omega}} - \frac{S_n^{\omega x} S_{n+1}^{\omega y}}{S_{n+1}^{\omega}} = S_n^{\omega xy} \\ &\quad + \omega_{n+1} x_{n+1} (y_{n+1} - m_{n+1}^{\omega y}) - \frac{S_n^{\omega x} S_{n+1}^{\omega y}}{S_{n+1}^{\omega}} \end{aligned} \quad (11)$$

The last term can be summed up the following way:

$$\begin{aligned} \frac{S_n^{\omega x} S_{n+1}^{\omega y}}{S_{n+1}^{\omega}} &= \left(\frac{1}{S_n^{\omega}} - \frac{\omega_{n+1}}{S_n^{\omega} S_{n+1}^{\omega}} \right) S_n^{\omega x} S_{n+1}^{\omega y} = \frac{S_n^{\omega x} S_{n+1}^{\omega y}}{S_n^{\omega}} \\ &\quad - \omega_{n+1} m_n^{\omega x} m_{n+1}^{\omega y} = \frac{S_n^{\omega x} S_n^{\omega y}}{S_n^{\omega}} \\ &\quad + \omega_{n+1} y_{n+1} \frac{S_n^{\omega x}}{S_n^{\omega}} - \omega_{n+1} m_n^{\omega x} m_{n+1}^{\omega y} \\ &= \frac{S_n^{\omega x} S_n^{\omega y}}{S_n^{\omega}} + \omega_{n+1} m_n^{\omega x} (y_{n+1} - m_{n+1}^{\omega y}) \end{aligned} \quad (12)$$

Thus

$$\begin{aligned} D_{n+1}^{\omega xy} &= S_{n+1}^{\omega xy} \\ &\quad + \omega_{n+1} x_{n+1} (y_{n+1} - m_{n+1}^{\omega y}) - \frac{S_n^{\omega x} S_{n+1}^{\omega y}}{S_{n+1}^{\omega}} \\ &\quad + \omega_{n+1} m_n^{\omega x} (y_{n+1} - m_{n+1}^{\omega y}) = \left[S_n^{\omega xy} - \frac{S_n^{\omega x} S_{n+1}^{\omega y}}{S_{n+1}^{\omega}} \right] \\ &\quad + \omega_{n+1} (x_{n+1} + m_n^{\omega x}) (y_{n+1} - m_{n+1}^{\omega y}) \\ &= D_n^{\omega xy} + \omega_{n+1} (x_{n+1} + m_n^{\omega x}) (y_{n+1} - m_{n+1}^{\omega y}) \end{aligned} \quad (13)$$

The following function written in C++ calculates the co-variation using Wellford's algorithm.

```
double WellfordCovariation(
    const std::vector<double>& x,
    const std::vector<double>& y) {
    double sumProducts = 0.;
    double xMean = 0.;
    double yMean = 0.;

    for (size_t i = 0; i < x.size(); ++i)
    {
        xMean += (x[i] - xMean) /
            (i + 1);
        sumProducts += (x[i] - xMean) *
            (y[i] - yMean);
        yMean += (y[i] - yMean) /
            (i + 1);
    }

    return sumProducts /
```

x.size();

Using the formula given in 13 we can derive the recurrent for covariation as follows:

$$\begin{aligned} C_{n+1}^{\omega xy} &= \frac{\sum_{i=1}^{n+1} \omega_i (x_i - m_{n+1}^{\omega x}) (y_i - m_{n+1}^{\omega y})}{\sum_{i=1}^{n+1} \omega_i} \\ &= \frac{D_{n+1}^{\omega xy}}{S_{n+1}^{\omega}} = \frac{1}{S_{n+1}^{\omega}} \cdot \left(D_n^{\omega xy} \right. \\ &\quad \left. + \omega_{n+1} (x_{n+1} + m_n^{\omega x}) (y_{n+1} - m_{n+1}^{\omega y}) \right) \\ &= \frac{D_n^{\omega xy}}{S_{n+1}^{\omega}} + \frac{\omega_{n+1}}{S_{n+1}^{\omega}} (x_{n+1} + m_n^{\omega x}) (y_{n+1} - m_{n+1}^{\omega y}) \end{aligned} \quad (14)$$

The first term can be summed up as follows

$$\begin{aligned} \frac{D_n^{\omega xy}}{S_{n+1}^{\omega}} &= \left(\frac{1}{S_n^{\omega}} - \frac{\omega_{n+1}}{S_{n+1}^{\omega} S_n^{\omega}} \right) D_n^{\omega xy} \\ &= \frac{D_n^{\omega xy}}{S_n^{\omega}} \left(1 - \frac{\omega_{n+1}}{S_{n+1}^{\omega}} \right) \\ &= C_n^{\omega xy} \left(1 - \frac{\omega_{n+1}}{S_{n+1}^{\omega}} \right) \end{aligned} \quad (15)$$

Thus

$$\begin{aligned} C_{n+1}^{\omega xy} &= C_n^{\omega xy} \left(1 - \frac{\omega_{n+1}}{S_{n+1}^{\omega}} \right) \\ &\quad + \frac{\omega_{n+1}}{S_{n+1}^{\omega}} (x_{n+1} + m_n^{\omega x}) (y_{n+1} - m_{n+1}^{\omega y}) \\ &= C_n^{\omega xy} \\ &\quad + \frac{\omega_{n+1}}{S_{n+1}^{\omega}} \left((x_{n+1} + m_n^{\omega x}) (y_{n+1} - m_{n+1}^{\omega y}) - C_n^{\omega xy} \right) \end{aligned} \quad (16)$$

The result is a formula that has a resemblance to the formula for updating the average.

IV. EXPERIMENTAL COMPARISON

For comparison, we wrote a program that implements ¹:

- Naive algorithm that directly calculates quantities $S_n^{\omega xy}$, $S_n^{\omega x}$ and $S_n^{\omega y}$.
- An algorithm using the Kahan method. [5]
- An algorithm using the Wellford's method.

The problem's data is formed as follows: two numbers are selected, m_x and m_y – the average of two samples. Then two more numbers are selected, d_x and d_y – respectively, the variances. The input to the algorithms is a sequence of numbers of the form

$$x_i = m_x \pm d_x; \quad y_i = m_y \pm d_y. \quad (17)$$

and the signs with deviations change at each iteration so that the true covariance is calculated as follows:

$$C_n^{xy} = \frac{\sum_{i=1}^n (x_i - m_x)(y_i - m_y)}{n} = d_x \cdot d_y \quad (18)$$

¹The code can be found in our GitHub repository <https://github.com/ashagraev/covariation>

True covariance is constant and does not depend on the number of terms so that we can calculate the relative calculation error for each method at each iteration. Therefore, in the current implementation $d_x = d_y = 1$, and the averages take the values of 100,000 and 1,000,000.

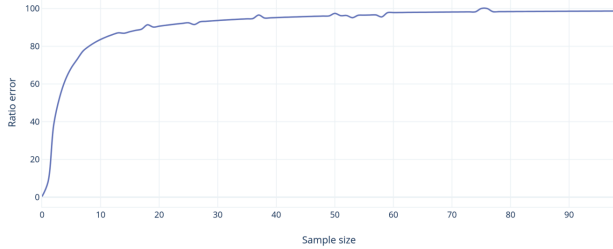


Fig. 1. Naive method ratio error

Fig. 1 shows the relative error dependence when using the naive method for an average of 100 thousand. This graph demonstrates the naive method's failure: at some point, the error begins to multiply, reaching completely unacceptable values. On the same data, the methods of Kahan and Wellford do not make significant errors.

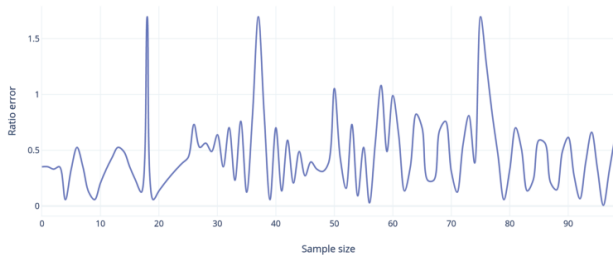


Fig. 2. Kahan method ratio error

Fig. 2 shows the results of the Kahan method, with an average of one million. The error does not grow with an increase in the number of terms, but although it is significantly lower than the error of the "naive" method, it is still too large for practical applications.

Wellford's method on fig. 3, in turn, and on this data demonstrates perfect accuracy .

V. CONCLUSION

This article compared several methods for calculating variance and covariance and made sure that the Wellford method gives the best results.

These methods allow saving much time and effort in situations where with data "poorly" location, i.e. it not aligned in memory, or it received online, or it requires storing a large number of metrics.

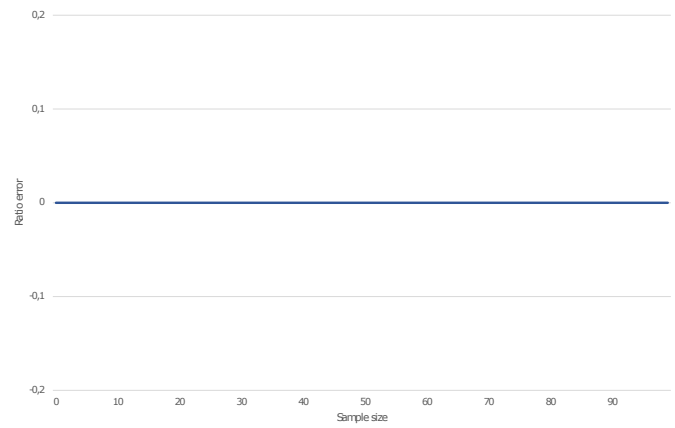


Fig. 3. Wellford's method ratio error

As shown in the article, unlike other approaches, this algorithm allows us to work with large amounts of data, which makes it useful in tasks related to Big Data [10]. Also, since the method allows to perform calculations quickly, it can be used to develop services related to high load [11].

One of the obvious possible applications is the methods associated with linear regression. Linear regression, despite its simplicity, can be an acceptable method of data mining in many cases.

Another application is the implementation of clustering algorithms, in particular, k-Means. Since these algorithms require constant calculation and storage of averages, as the number of objects increases, the cost of storing averages will increase by this number.

Summing up, we can say that the article presents a simple and understandable method for calculating the main statistical characteristics with zero error and high speed, which will be important in big data and machine learning.

ACKNOWLEDGMENT

This work was supported by RFBR, project number 20-07-00498, 18-01-00548 and 18-29-03088.

REFERENCES

- [1] S. Liao, J. Li, Y. Liu, Q. Gao, and X. Gao, "Robust formulation for pca: Avoiding mean calculation with l 2, p-norm maximization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [2] G. Munaò, A. Correa, A. Pizzirusso, and G. Milano, "On the calculation of the potential of mean force between atomistic nanoparticles," *The European Physical Journal E*, vol. 41, no. 3, pp. 1–9, 2018.
- [3] N. Bezak, L. Cerović, and M. Šraj, "Impact of the mean daily air temperature calculation on the rainfall-runoff modelling," *Water*, vol. 12, no. 11, p. 3175, 2020.
- [4] B. Wellford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

- [5] W. Kahan, "Further remarks on reducing truncation errors, commun," *Assoc. Comput. Mach.*, vol. 8, p. 40, 1965.
- [6] S. P. Sauer, J. R. Sabin, and J. Oddershede, "Calculation of mean excitation energies," *Advances in Quantum Chemistry*, vol. 80, pp. 225–245, 2019.
- [7] Q. Dai, Y. Huang, L. Hao, and W. Cai, "Calculation and measurement of mean room surface exitance: the accuracy evaluation," *Lighting Research & Technology*, vol. 51, no. 6, pp. 956–968, 2019.
- [8] T. F. Chan, G. H. Golub, and R. J. LeVeque, "Algorithms for computing the sample variance: Analysis and recommendations," *The American Statistician*, vol. 37, no. 3, pp. 242–247, 1983.
- [9] machinelearning.ru. (2020) The addition of a large amount of numbers, significantly different in size.
- [10] M. Toghraee, "Calculation of mean data on gini relationship by data mining method," *nature*, vol. 8, p. 10, 2019.
- [11] B. Lopez, A. Mahvash, M. G. Lam, and S. C. Kappadath, "Calculation of lung mean dose and quantification of error for 90y-microsphere radioembolization using 99mtc-maa spect/ct and diagnostic chest ct," *Medical physics*, vol. 46, no. 9, pp. 3929–3940, 2019.